ME 3204: Mechatronics Design Project Report

Automated Smart Barrier Gate System

By: Akinfoluhan Akinleye

Table of Contents

System Description ……3

System Components ...……4

New Elements ...……..6

Appendix A ……7

Appendix B ……...16

Appendix C ……...17

Appendix D ……...19

Appendix E ……...20

System Description

My project is an automated smart barrier gate system inspired by the parking lot next to my apartment, since I see a barrier gate there every day. The goal was to develop a smaller prototype of a real controlled-entry system that could detect a vehicle, verify access, and open or deny entry based on programmed conditions. The system uses an ultrasonic sensor to detect when a vehicle is present in front of the gate. Once a vehicle is detected, the LCD prompts the user to scan an RFID card. If the card is authorized, the servo motor opens the gate and the system displays messages such as "Access Granted," "Please Proceed," and "Gate Closing." If the card is not authorized, the system displays "Access Denied" and keeps the gate closed. In addition to the main gate function, I also added a passive buzzer, an IR remote override, and a stepper motor that continuously rotates a banner on top of the structure. The final result is a working mechatronics prototype that combines sensing, control, motion, and user feedback through components including an ultrasonic sensor, RFID reader, servo motor, and stepper motor.

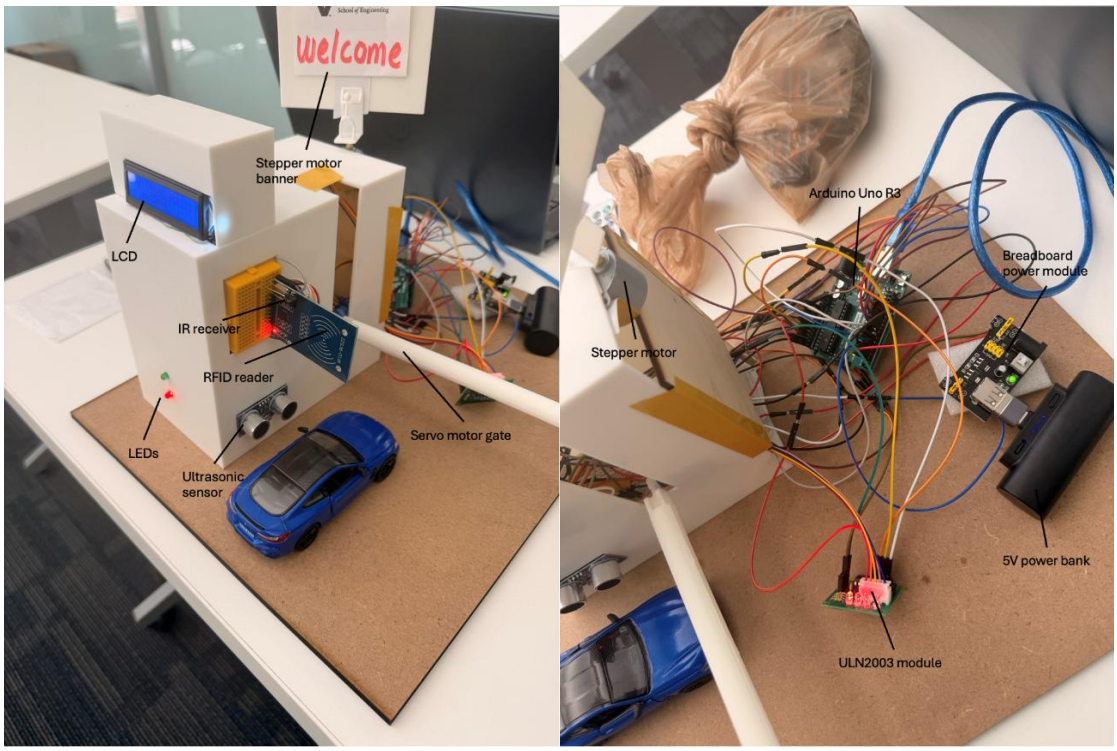


Figure 1. Final fabricated smart barrier gate system showing the main exterior components and the rear electronics/power arrangement. The front view identifies the LCD, RFID reader, IR receiver, LEDs, ultrasonic sensor, servo-driven gate arm, and stepper motor banner, while the rear view shows the Arduino Uno, ULN2003 stepper driver, breadboard power module, and external 5 V power bank used for the stepper motor system.

System Components

The system is controlled by an Arduino Uno, which serves as the main processing unit of the project. All input devices send signals to the Arduino, and the Arduino processes those signals and commands the output devices. The main input devices in the system are the ultrasonic sensor, RFID reader, and IR receiver. The main output and actuator devices are the servo motor, LCD, LEDs, passive buzzer, and stepper motor. What made this project interesting was not just using these parts individually, but getting them all to work together in one coordinated system.

The first stage of operation is vehicle detection. I mounted the ultrasonic sensor so it faces the approaching vehicle area in front of the barrier arm. The Arduino sends a trigger pulse to the sensor and reads the echo time to determine the distance of an object. If the measured distance is below a selected threshold, the program recognizes that a vehicle is present. Once that happens, the LCD displays a prompt telling the user to scan a card.

The next major part of the system is the RFID access control. I added an RC522 RFID reader and programmed the system to recognize the hexadecimal UID values of authorized cards. Before integrating this into the full system, I first had to run a separate program to read the UID values of each card and tag. After collecting those values, I inserted them into the full project code so the Arduino could compare scanned cards against the authorized list. If the scanned card matches one of the stored UID values, access is granted. If not, access is denied. This part of the build was one of the most challenging because the RFID reader worked on its own at first, but became unreliable when connected as part of the larger system with the servo motor and LCD. I had to troubleshoot the power setup carefully. I first used a voltage divider with a 5k and 10k resistor combination to get about 3.3 V, but the RFID behavior was still inconsistent and sometimes it would detect the card without reading the hexadecimal value. I eventually adjusted the divider to produce about 3.57 V using two 2k resistors and a 10k resistor, and that gave me more consistent readings. This was one of the biggest debugging steps in the project and forced me to think more carefully about electrical behavior, not just code.

The servo motor acts as the main gate actuator. Once access is granted, the Arduino sends the servo to the open position so the barrier arm lifts. After a programmed delay, the system checks if the vehicle is still present. If it is, the LCD displays "Please Proceed," waits a little

longer, and then closes the gate. If access is denied, the servo remains in the closed position. I also used red and green LEDs to provide quick visual feedback. The green LED signals a successful access state, while the red LED represents the closed or denied state. The passive buzzer adds another level of feedback by creating different tones for granted access, denied access, and gate closing. Coding the passive buzzer took more effort than an active buzzer because it required a different signal approach and had to be integrated carefully with the rest of the system.

I also added an IR remote control feature using an IR receiver and remote. Like the RFID reader, this required a separate test program first so I could determine the hexadecimal signal from each remote button and choose the one I wanted to use as the override command. This gave the system a wireless manual override feature, which made it feel more realistic and more advanced than a simple gate that only depends on one input.

Another major addition was the rotating banner on top of the structure, driven by a stepper motor and ULN driver module. This banner moves continuously the entire time the system is powered on, regardless of whether the gate is opening, closing, or idle. Adding the stepper motor introduced both wiring and power challenges. At one point, the motion of multiple components became jerky, and I initially tried to solve that through code. Later, I realized the real issue was the electrical load placed on the Arduino 5 V line by the servo, RFID reader, LCD, stepper motor, and other devices together. That pushed me to think more like an engineer and investigate the cause instead of only adjusting the program. I eventually used a phone power bank with a breadboard power module to supply 5 V separately for the stepper motor system, while still maintaining a common ground with the Arduino. That made the independent banner motion much more stable.

The physical construction itself was more straightforward than the electronics and coding because I created a CAD design and had the main structure 3D printed. The only change I had to make afterward was drilling holes for the LEDs because I forgot to include them in the original design. Even though the physical design process was smoother than the electronics and coding, it still required planning, especially because I needed enough long male-to-male and female-to-female jumper wires to connect the Arduino pins on the outside of the structure to the

components mounted inside. This project taught me the importance of planning materials ahead of time, especially when designing a system where appearance and internal wiring both matter.

Overall, almost every part of the project failed the first time except the LEDs. That forced me to debug step by step and improve both my coding and my patience. By the end of the project, I had a much better understanding of Arduino programming, wiring, troubleshooting, and system integration. I also gained a better appreciation for how hardware limitations and power distribution can affect system performance.

New Elements

- RFID reader and UID-based authorization: This added real access control and required separate RFID testing before full integration.
- IR remote override: This created a wireless override function and required separate code to identify the desired remote button signal.
- Passive buzzer: This provided audio feedback and required different coding than an active buzzer.
- Stepper motor with ULN driver and rotating banner: This added a second actuator system running independently of the barrier gate.
- Breadboard power module and power bank integration: This was added to create a more stable power arrangement for the stepper motor after I identified load-related issues.
- Functional construction: The final housing was designed in CAD, 3D printed, and built to support the actual positions of the sensors, motors, and display rather than just serving as decoration.

Appendix A. Detailed Replication Procedure

This appendix explains how to recreate the automated smart barrier gate system from the beginning. The goal is to provide enough detail for another student to rebuild the project, understand how the subsystems interact, and test each stage before integrating the full design.

A1. Project Purpose and General Operation

The purpose of this project is to simulate a smart parking or checkpoint barrier gate system. The completed system detects a vehicle with an ultrasonic sensor, prompts the user to scan an RFID card, checks whether the scanned card is authorized, and then either opens or denies access. If access is granted, a servo motor raises the barrier arm, the LCD displays status messages, LEDs and a buzzer provide user feedback, and the gate closes after a programmed delay. An IR remote can also override the system manually. In addition, a stepper motor continuously rotates a banner while the system is powered.

To replicate the project successfully, it is best to build and test the system in stages instead of wiring every component at once.

A2. Required Materials

The following components were used in the final project:

- Arduino Uno R3
- RC522 RFID reader with RFID card and tag
- HC-SR04 ultrasonic sensor
- 16x2 LCD display with I2C interface
- Servo motor for the barrier arm
- Passive buzzer
- Red LED
- Green LED
- IR receiver

- IR remote
- 28BYJ-48 stepper motor
- ULN2003 stepper motor driver module
- Breadboard power module
- 5 V power bank
- Breadboard
- Jumper wires, including long male-to-male and female-to-female wires
- Two 220Ω Resistors for LEDs
- 3D-printed structure and base
- Barrier arm material
- Rotating banner mounted to the stepper motor
- Adhesive, tape, or mounting supports as needed

The final prototype also required extra jumper wires because many of the components were mounted on the exterior of the structure while the Arduino and most wiring were kept toward the back.

A3. Physical Construction of the Prototype

The structure of the project was designed first in CAD and then 3D printed. The design used a raised front enclosure to hold the visible user interface components and a rear open area to allow access to the Arduino, power module, and additional wiring.

To replicate the physical structure:

1. Create or print a main enclosure tall enough to support the LCD, RFID reader, ultrasonic sensor, LEDs, and the internal servo motor.
2. Leave an opening in the front upper section for the LCD.

3. Leave a mounting position on the side/front for the RFID reader so that the card can be scanned externally.
4. Mount the ultrasonic sensor on the lower front face so it points toward the vehicle approach area.
5. Drill or include two small holes for the red and green LEDs.
6. Mount the servo motor inside the enclosure so the barrier arm extends outward through the side.
7. Add a rear or top support section for the stepper motor and rotating banner.
8. Keep the back or top at least partly accessible so wiring can be adjusted during testing.

In my build, the only physical design correction I had to make after printing was drilling holes for the LEDs because I forgot to include them in the CAD file.

A4. Subsystem-by-Subsystem Electrical Build Procedure

The project should be assembled in stages. Testing each stage before moving to the next reduces debugging time.

A4.1 LED Test

Begin with the simplest outputs first.

1. Connect the red LED to its Arduino output pin through a resistor.
2. Connect the green LED to its Arduino output pin through a resistor.
3. Connect both LED grounds to Arduino ground.
4. Upload a basic LED blink sketch to confirm both LEDs function correctly.

This stage confirms that the Arduino is powered correctly and that the output pins are working.

A4.2 Servo Motor Test

Next, test the barrier gate actuator.

1. Mount the servo motor in the enclosure.

2. Attach the barrier arm securely to the servo horn.
3. Connect the servo signal wire to the assigned Arduino pin.
4. Connect servo power and ground.
5. Upload a simple servo test program that moves the servo between two angles.
6. Adjust the open and closed angles until the barrier arm moves to the correct positions.

This step is important because the physical range of motion depends on how the arm is mounted.

A4.3 Ultrasonic Sensor Test

After the actuator is working, test the first sensor.

1. Mount the ultrasonic sensor in the front lower section of the structure.
2. Connect its VCC, GND, trigger, and echo pins.
3. Upload a distance-reading program.
4. Use the serial monitor to confirm the sensor can detect the toy car at the intended distance.
5. Decide on a distance threshold for vehicle presence.

This threshold will later be used in the final program to trigger the “Scan Card” stage.

A4.4 LCD Test

Next, test the display.

1. Mount the 16x2 LCD in the front opening of the structure.
2. Connect VCC, GND, SDA, and SCL.
3. Upload a simple LCD test sketch.
4. Confirm that text appears and is readable from the front of the model.
5. If the display does not work, verify the I2C address and adjust the code if necessary.

This step ensures the system can provide readable status messages to the user.

A4.5 RFID Test

The RFID reader should be tested separately before being integrated into the full system.

1. Mount the RC522 RFID reader on the outside wall of the structure so cards can be scanned easily.
2. Connect the RC522 pins to the Arduino using the SPI pin arrangement used in the final design.
3. Upload an RFID test sketch that reads card or tag UIDs.
4. Scan each card and tag individually.
5. Record the hexadecimal UID values of every RFID card or tag to be authorized.

This stage is essential because the final integrated code cannot work until the correct UID values are known.

In my project, RFID was one of the hardest parts to stabilize. It worked by itself at first, but when I connected it together with the servo motor and LCD, it became inconsistent. Sometimes it detected a card without reading the hexadecimal UID. To solve this, I adjusted the RFID supply using a resistor-divider-based voltage setup derived from the Arduino 5 V line. After experimenting with values, I obtained more consistent results with a slightly increased voltage level. This was one of the most important debugging stages in the project.

A4.6 IR Receiver and Remote Test

The IR receiver should also be tested separately before being merged into the main program.

1. Mount the IR receiver so it faces outward and is visible to the remote.
2. Connect the IR receiver signal, VCC, and GND.
3. Upload an IR test sketch.
4. Press the buttons on the remote one at a time.
5. Record the hexadecimal command values shown in the serial monitor.
6. Select the desired button to use as the manual override in the final project.

This stage makes it possible to define the correct remote command in the full system code.

A4.7 Passive Buzzer Test

1. Connect the passive buzzer to its assigned output pin and ground.
2. Upload a simple tone test sketch.
3. Confirm that different tones can be produced.

This is important because the passive buzzer requires tone-based coding and behaves differently from an active buzzer.

A4.8 Stepper Motor and ULN2003 Test

The stepper-driven banner should be built and tested as a separate subsystem before being integrated.

1. Mount the stepper motor at the top of the structure.
2. Attach the rotating banner securely to the motor shaft.
3. Connect the stepper motor to the ULN2003 driver board.
4. Connect the ULN2003 control pins to the Arduino.
5. Provide separate 5 V power to the stepper motor system using the power bank and breadboard power module.
6. Connect the ground of the external 5 V supply to the Arduino ground so that both systems share a common ground.
7. Upload a test sketch that rotates the stepper motor back and forth by approximately 180 degrees.
8. Confirm that the banner rotates continuously and smoothly.

This part required special attention to power distribution. When too many components were powered from the Arduino 5 V line, the motion became jerky. I initially tried to solve that through code, but later realized the issue was electrical loading. The final solution was to power

the stepper motor system separately using a power bank and breadboard power module while keeping a common ground with the Arduino.

A5. Final Pin and Power Integration

After each subsystem works on its own, combine them into the full system.

1. Connect all final sensor and output wires to the Arduino according to the final pin table.
2. Verify that all grounds are common where required.
3. Keep the stepper motor power separate from the main Arduino-driven sensor/control section.
4. Place the Arduino and most wiring at the rear of the model.
5. Route long wires through or behind the enclosure to reach the front-mounted components.
6. Secure loose wires as much as possible so they do not interfere with the barrier arm or banner movement.

At this stage, the project should physically resemble the final prototype and should contain all sensors, outputs, and actuators at once.

A6. Software Integration Procedure

Once all subsystems have been tested individually, the final code can be uploaded.

1. Install the required Arduino libraries for:
    - RC522 RFID reading
    - LCD display
    - IR receiver
    - Servo control
    - Stepper motor control
2. Insert the recorded RFID hexadecimal UID values into the main code.

3. Insert the selected IR remote command into the main code.
4. Confirm the ultrasonic threshold value is appropriate for the physical placement of the vehicle.
5. Upload the final integrated code.

The final integrated code should perform the following sequence:

1. Rotate the banner continuously.
2. Display the idle system message.
3. Detect a vehicle with the ultrasonic sensor.
4. Prompt the user to scan an RFID card.
5. Verify whether the scanned card is authorized.
6. Grant or deny access.
7. Open the barrier gate if access is granted.
8. Display “Please Proceed” if the vehicle is still present after the initial open delay.
9. Close the barrier gate after the programmed delay.
10. Allow remote override independently of RFID.

A7. Final Testing Procedure

After uploading the full code, test the final system in this order:

1. Power the Arduino and the separate stepper motor system.
2. Confirm that the stepper motor banner begins rotating immediately.
3. Confirm that the LCD shows the idle message.
4. Place the toy car in front of the ultrasonic sensor.
5. Confirm that the LCD changes to the “Scan Card” message.
6. Scan an authorized RFID card.

7. Confirm that the gate opens, the LEDs change correctly, and the buzzer sounds.
8. Confirm that the “Please Proceed” message appears only when the car is still detected after the initial open time.
9. Confirm that the gate closes after the extra delay.
10. Scan an unauthorized RFID card and confirm that access is denied.
11. Test the IR remote override and confirm that it opens the gate without RFID scanning.
12. Let the full system run for several cycles to check stability.

A8. Important Replication Notes

A few specific observations from the build should be noted for anyone trying to reproduce the project:

- The RFID reader may behave differently when tested alone versus when integrated with other powered components.
- Long jumper wires may be necessary if the structure places components on the outside while the Arduino stays inside or at the rear.
- Power distribution matters. If too many moving components share the same 5 V source, motion may become unstable.
- The stepper motor system works more reliably when powered separately from the Arduino logic side.
- The project is much easier to debug if each subsystem is tested alone first before running the full integrated program.
- Good wire planning matters. Even if the logic is correct, poor wire routing can make the project harder to assemble and troubleshoot.

Appendix B. Pin Connection Table

Component	Connection
Red LED	D2
Passive buzzer	D3
Green LED	D4
Servo motor signal	D5
Ultrasonic sensor TRIG	D6
Ultrasonic sensor ECHO	D7
Stepper motor driver IN1	D8
RFID reader RST	D9
RFID reader SDA/SS	D10
RFID reader MOSI	D11
RFID reader MISO	D12
RFID reader SCK	D13
IR receiver output	A0
Stepper motor driver IN2	A1
Stepper motor driver IN3	A2
Stepper motor driver IN4	A3
LCD SDA	A4
LCD SCL	A5

Power and ground notes

- The servo motor, ultrasonic sensor, LCD, IR receiver, buzzer, LEDs, and main control logic were connected to the Arduino-based control system.
- The RFID reader required a custom resistor-divider-based voltage arrangement derived from the Arduino 5 V line in order to achieve reliable card reading in the final integrated system.
- The stepper motor was connected to the ULN2003 driver board.

- The stepper motor system was powered separately using a 5 V phone power bank connected through a breadboard power module.
- The external power system ground and Arduino ground were tied together to create a common ground reference.

Appendix C. Parts List

Part	Quantity	Purpose in Project
Arduino Uno R3	1	Main controller
RC522 RFID reader	1	Reads RFID card/tag for access control
RFID white card	1	Authorized access card
RFID key tag	1	Authorized access tag
Ultrasonic sensor	1	Detects vehicle presence
16x2 I2C LCD	1	Displays system messages
Servo motor	1	Raises and lowers the barrier gate
Red LED	1	Closed / denied status indicator
Green LED	1	Open / granted status indicator
Passive buzzer	1	Audio feedback
IR receiver	1	Receives remote override signal
IR remote	1	Wireless override input
28BYJ-48 stepper motor	1	Rotates the banner continuously
ULN2003 driver board	1	Drives the stepper motor

Breadboard power module	1	Selects and distributes separate 5 V power for the stepper system
5 V phone power bank	1	External power supply for the stepper motor system
Breadboard	1	Wiring and connection support
Jumper wires	many	Connect Arduino, modules, and power
Resistors for LEDs	2	Current limiting for red and green LEDs
Resistors for RFID voltage divider	several	Adjusted RFID supply voltage in final build
3D-printed enclosure / structure	1	Physical housing for system
Barrier arm material	1	Physical gate arm
Banner material	1	Rotating top banner

Additional material notes:

A large number of long jumper wires were needed because several components were mounted on the outside of the 3D-printed structure while the Arduino and much of the wiring were located toward the back of the model. Additional resistors were also used during debugging of the RFID power setup.

Appendix D. Arduino Code

D1. Final Integrated Project Code

This code controls the complete smart barrier gate system. It integrates the ultrasonic sensor, RFID reader, LCD, servo motor, LEDs, passive buzzer, IR remote override, and the continuously rotating stepper motor banner into one program.

Google Doc / Drive Link:

Link

D2. RFID UID Test Code

This code was used before full integration to read the hexadecimal UID values of the RFID card and tag. These values were then inserted into the final project code so the system could determine which RFID inputs were authorized.

Google Doc / Drive Link:

Link

D3. IR Receiver Test Code

This code was used before full integration to identify the hexadecimal command values produced by the remote control buttons. After testing, one button value was selected and used in the final system as the remote override command.

Google Doc / Drive Link:

Link

Appendix E. Additional Figures and Construction Notes

E1. Final System Photographs

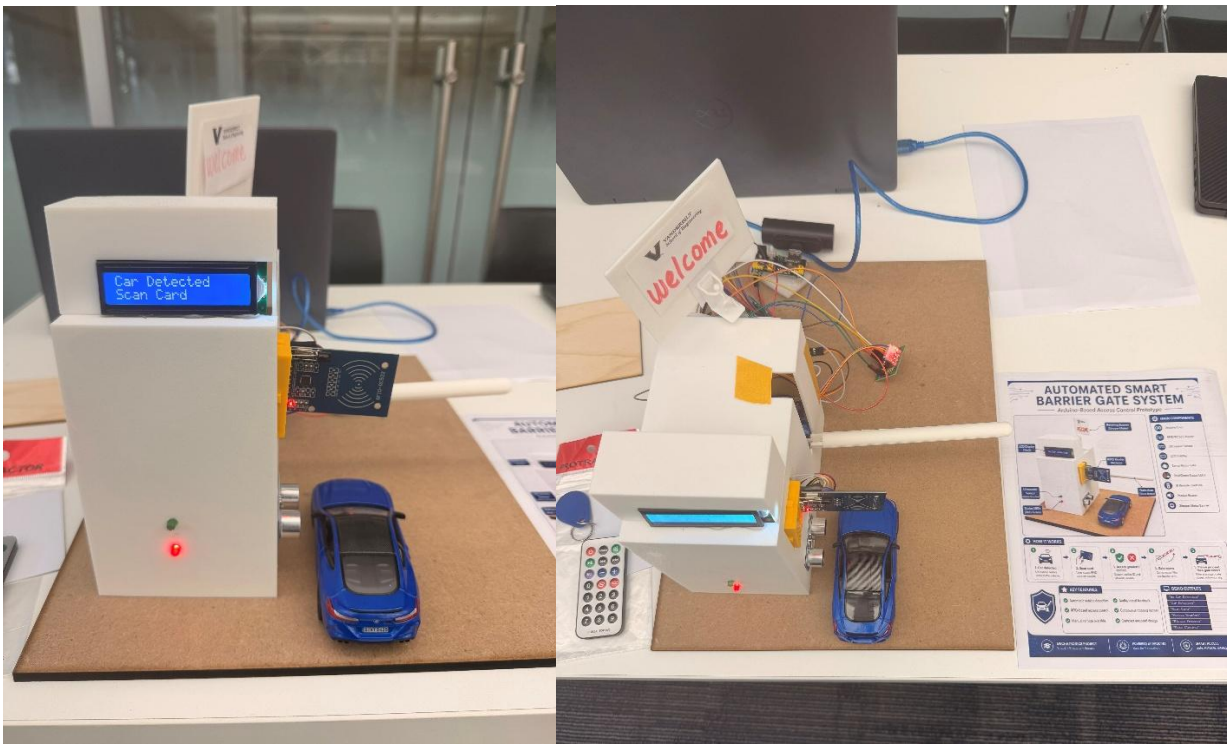


Figure E1. Front and Top views of the final fabricated smart barrier gate system showing the LCD, RFID reader, LEDs, ultrasonic sensor, servo-driven gate arm, and rotating banner.

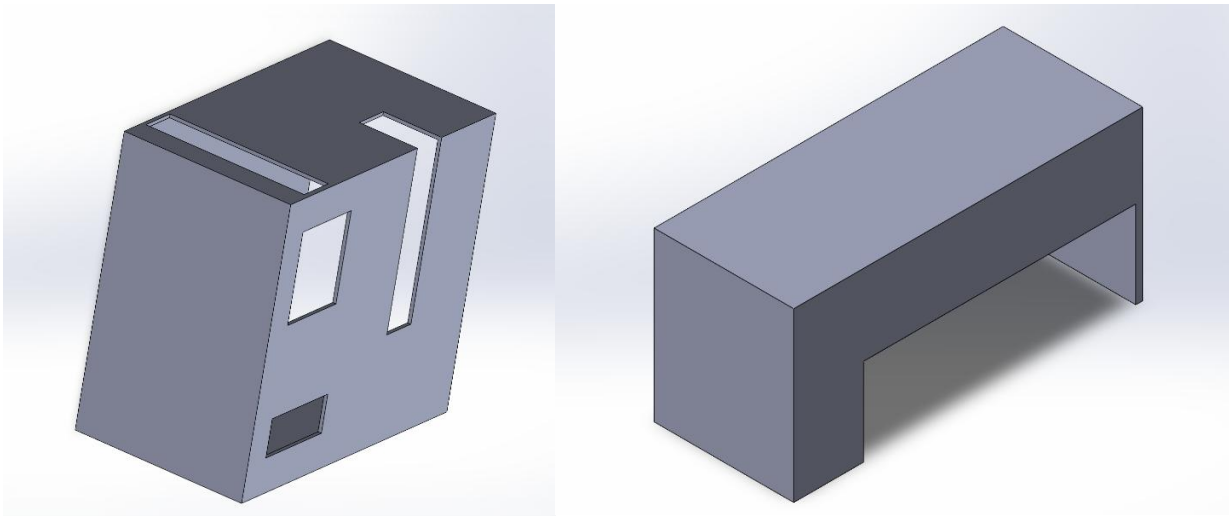


Figure E2. CAD views of the enclosure components used to fabricate the final smart barrier gate prototype. The left view shows the main front housing designed to support the LCD, RFID reader, ultrasonic sensor, and internal gate mechanism, while the right view shows the upper support structure used for the rotating banner assembly.

E2. Construction Notes

The final structure of the project was designed in CAD and then 3D printed. The enclosure was made to support the LCD, RFID reader, ultrasonic sensor, LEDs, servo motor, and rotating banner while still leaving room in the rear for the Arduino and wiring. The LCD was mounted in the upper front opening of the structure. The RFID reader was mounted on the side/front area so that the card could be scanned from outside. The ultrasonic sensor was placed on the lower front section so it could face the approaching car. The servo motor was mounted inside the structure and connected to the barrier arm. The stepper motor and banner were mounted at the top. After printing, holes had to be drilled for the LEDs because they were not included in the original CAD design.

E3. Wiring Notes

Most of the wiring was routed from the rear of the system to the front-mounted components. Long jumper wires were required because the Arduino and several electrical connections were located behind the visible front face of the structure while the sensors and output devices were mounted on the exterior. The RFID reader, LCD, ultrasonic sensor, LEDs, IR receiver, servo motor, and buzzer were all connected to the Arduino. The stepper motor was connected to the ULN2003 driver module. The stepper motor system was powered separately using a 5 V phone power bank connected to a breadboard power module. The Arduino ground and the external power ground were tied together to create a common ground reference. This was necessary so that the stepper motor could operate reliably while the Arduino continued to control the rest of the system.

E4. Replication Notes

Anyone attempting to reproduce this project should test each subsystem separately before integrating the entire design. The RFID reader should be tested first with a separate code to determine the hexadecimal UID values of the authorized card and tag. The IR receiver should also be tested separately to determine the hexadecimal output of the selected remote button. The

servo motor should be tested alone first so the correct open and closed angles for the barrier arm can be chosen. The ultrasonic sensor should be tested alone to determine the appropriate vehicle-detection threshold. The stepper motor and ULN2003 driver should also be tested separately before being added into the full program. During final integration, careful attention should be given to wiring length, wire routing, and power distribution. The project is easier to build and debug when each subsystem is confirmed to work on its own before the final combined code is uploaded.